Infrastructure from Code: Nitric, Encore and a Python Prototype

Giacomo Pizzamiglio

DARE 2026

1 Introduction

A cloud application is usually described twice. The code holds the handlers and the calls that read an object or publish a message. The infrastructure files declare the buckets, queues and functions, and the permissions the code needs. The two are written, reviewed and deployed separately. Infrastructure as Code (IaC) keeps the second description in versioned files that a provisioning engine applies [1, 2, 3, 4], but those files say nothing about the code that uses the resources.

The IaC workflow does not check that the two descriptions agree, so they drift apart when the application changes. A handler gains a call that writes to a second bucket, the permission is not added, and the call fails at run time. In the other direction, a call is removed and its permission stays, and nothing fails. Identity and access management (IAM), and least privilege in particular, is described as a key challenge of IaC [1].

Infrastructure from Code (IfC) derives the infrastructure configuration from the application code, which becomes the single source of truth [5, 6]. A tool can learn what the application needs in two ways. With *inference*, it derives resources and permissions from the code, by analyzing SDK calls or framework annotations. With *declaration*, the developer states in the code a resource, or how a component uses it, and the tool collects these statements. Declarations are explicit and easy to analyze, but they sit next to the calls they describe and can drift from them in the same way as infrastructure files.

This report asks whether, and how, IfC keeps application code and infrastructure aligned when the application changes. I compare Nitric and Encore on one application and a set of changes to its code. I then build pyifc, a minimal IfC tool for Python that checks declarations against the code before it generates per-component permissions, and run it through the same changes.

2 Infrastructure from Code Tools

Several products have been presented as IfC [7], and a 2025 article reports only Nitric and Encore as still active [8]. I considered five tools. Wing is a language for cloud applications that is pre-release and community-maintained [9, 10]. Klotho, which generated Pulumi code for AWS from comment annotations, has been discontinued [11, 8]. Shuttle, which provisioned resources on its own hosted platform from attribute macros, has ceased operations [12, 13]. Nitric and Encore are active frameworks in which resources are declared as SDK objects in the code. They differ in two respects: whether access comes from a separate declaration or is derived from the code that uses a resource, and whether the tool runs the code or only parses it. I ran both, Nitric with CLI 1.61.1 and Encore with CLI 1.58.6.

Nitric. Nitric is an open-source IfC framework [14]. With its SDK the developer declares the resources (APIs, buckets, topics) and the access each service needs, for example `bucket("assets").allow("read")`. On deployment the CLI builds one container image per service and runs each service in a collection mode that records the declarations. They form

the model of the application, which a provider plugin turns into Terraform code or into cloud resources on AWS, Google Cloud or Azure [15].

Encore. Encore is an open-source backend framework for Go and TypeScript with an optional commercial cloud platform [16]. Resources are declared at package level by calling typed constructors with a literal name, such as `objects.NewBucket("staging", ...)`, and endpoints are marked with annotations. A service uses a resource directly or through a typed reference that names the operations it needs, such as `objects.BucketRef[objects.Uploader]`. A parser reads the source without running it and builds an Application Model of services, endpoints, resources, and which service uses which resource and how. The model drives the local run, the build-time checks, the permissions and the deployment: a service that only downloads from a bucket gets read access. Encore Cloud provisions the resources in the user's own AWS or Google Cloud account [16]; for self-hosting, Encore builds a Docker image and generates no Terraform [17].

Table 1 compares the two tools, from their documentation and source code.

	Nitric	Encore
Declared	Resources and per-service access, as SDK calls. Cloud and region in a stack file. Not in code: cloud settings, resource options.	Resources and endpoints, as typed SDK objects with a literal name, and annotated functions. Access only as the operation types of a reference. Not in code: IAM, instance sizes, regions.
Inferred	Invoke rights for gateways, topics, schedules and other triggers, collected in the collection mode. Access is not derived from what the handlers call.	Which service uses which resource and how, and the permissions that follow, by static analysis. The rules for the permissions are not documented.
Resources	APIs, WebSockets, schedules, batch jobs, topics, queues, buckets, SQL, key/value stores, secrets, websites. Node.js, Python, Go; AWS, Google Cloud, Azure; some are Preview.	Services, APIs, PostgreSQL, topics and subscriptions, buckets, Redis caches, cron jobs, secrets. Go and TypeScript; Encore Cloud on AWS and Google Cloud. No queues.
Beyond the tool	Stack-file settings, Dockerfile template, custom provider; direct cloud SDK calls get no Nitric permissions.	Raw endpoints, infrastructure outside the model, self-hosting with an infra config, import of existing resources. No way to override the derived permissions is documented.
Local development	`nitric start`: services as processes against local stand-ins, with a dashboard.	`encore run`: local resources and a dashboard; PostgreSQL in Docker; cron jobs do not run.

Table 1: Nitric and Encore: what the developer declares, what the tool infers, the resources it supports, how to go beyond it, and how it runs locally.

Each design has its own way of going wrong when the code changes. A separate declaration can go stale: a use may lack its permission, or a permission may remain after its use is removed. Access derived from the code changes whenever the code does, with no separate step at which the change is reviewed. The scenarios in Section 3 exercise both.

3 Sensor Hub and Change Scenarios

3.1 Sensor hub

The tools are compared on one application, a train sensor hub. An operator sends a JSON reading to `POST /readings`. A reading has the kind `speed` or `line_voltage`, two features exposed by the train API of Siemens Railigent X [18]. Ingestion stores the original JSON in a staging bucket and publishes a job reference to the topic of its kind. One parser per kind consumes its topic, reads the staged object and writes a normalized record to a results bucket. The query component

serves `GET /readings`, which lists the processed records, and a dashboard polls it (Figure 1). Each tool has its own port of the application, in Python for Nitric and in Go for Encore, which runs on the tool's own local runtime. Both ports use topics for the two channels: Encore has no queues, and in Nitric's Python SDK a queue cannot trigger a worker.

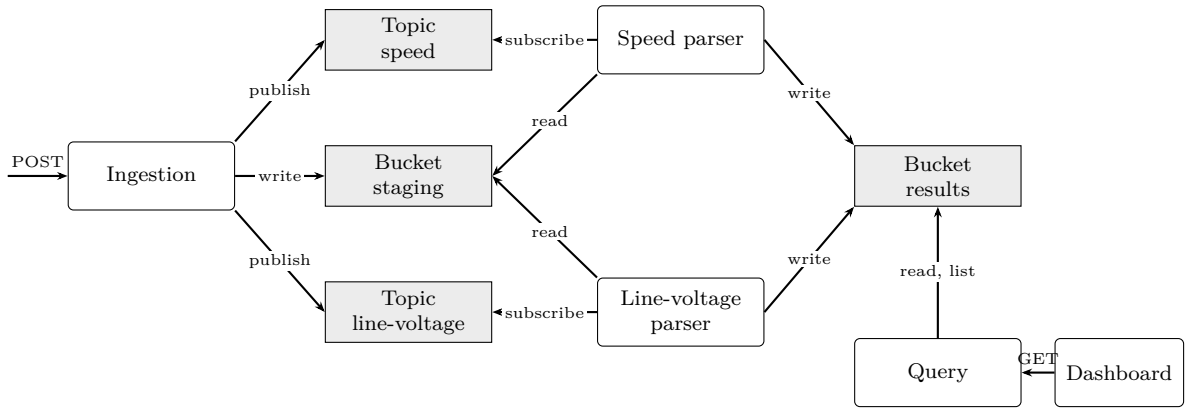


Figure 1: The sensor hub. Rounded boxes are components, shaded boxes are resources, and arrows go from a component to the resource it uses, labeled with the capability: ten uses of four resources, where listing counts as reading.

3.2 Change scenarios

I changed the code of each port in eight ways (Table 2). The changes cover the two directions of drift from the introduction: a use without a permission (T2, T2b), and a permission or subscription with no counterpart in the code (T4, T5). T6 renames a stateful resource, and T0, T1 and T3 check that changes made consistently in code and declarations reach the model.

ID	Kind	Change
T0	Control	No change; generate again
T1	Follow	Add a third bucket `archive`; ingestion writes to it
T2	Detect	Query writes to staging without having declared that access
T2b	Detect	Query writes to staging through a handle declared read-only
T3	Follow	Add `archive`, then remove the write call and its permission together; the bucket stays declared
T4	Detect	Add `archive`, remove only the write call, keep the permission
T5	Detect	Add a topic `audit` with a subscriber and no publisher
T6	Protect	Rename the staging bucket to `staging-v2`

Table 2: Change scenarios. T3 and T4 leave the same code; only the leftover permission differs. In Encore the access of a service is the type of reference it takes, so T2b is the same edit as T2.

Each tool starts from a baseline: the Terraform it generates from the unchanged port, applied once to Floci, an AWS emulator. A test harness applies each change to a copy of the port and records the tool's model, its export, the plan and all tool output. The *model* lists the resources and how each component uses them (`nitric spec`, `encore debug meta`). The *export* is the Terraform the tool generates; Encore generates none, so its export comes from a compiler I built on its model. The *plan* is what `terraform plan` reports for the export against the baseline, without the changes that Terraform's Docker provider shows even when the baseline is planned against itself. The tool output includes a smoke test on the local runtime.

Model, export and plan are compared with the *reference*: the ten uses of Figure 1 with those the change adds or removes. A *signal* is a failed step, a warning or error line that the unchanged

port does not produce, or a traceback or HTTP 5xx error in the run log. It can appear at four stages: edit (`mypy` for Nitric, `encore test` for Encore), generation, plan and run time. Each kind of scenario has its own pass rule. *Follow* passes when the model, and the export where captured, match the reference. *Control* passes when the model matches the reference and the plan has no changes. *Detect* passes when the model matches the reference and no topic has a subscriber without a publisher, or when the tool raises a signal. *Protect* passes when the plan does not destroy or replace a bucket, or when the tool raises a signal at edit or generation. A follow or control scenario that does not pass *fails*; a detect or protect scenario that does not pass is *missed*. The rules measure whether the model follows the code; they do not judge whether a change in access is wanted.

4 Results for Nitric and Encore

Table 3 gives the verdicts. Both tools pass the control and follow scenarios. Of the five detect and protect scenarios, Nitric passes one and Encore two, and the rest are missed. All four signal stages ran in every case, so in each missed case no stage raised a signal.

ID	Kind	Nitric	Encore
T0	Control	pass	pass
T1	Follow	pass	pass
T2	Detect	pass (signal at run time)	pass (model follows the code)
T2b	Detect	missed	pass (model follows the code)
T3	Follow	pass	pass
T4	Detect	missed	missed
T5	Detect	missed	missed
T6	Protect	missed (bucket deleted in the plan)	missed (bucket deleted in the plan)

Table 3: Verdict per scenario and tool.

Declared changes (T0, T1, T3). Both tools carry a new bucket and its permission from the code to the model and the export. When the write and its permission are removed together, the export creates the bucket and no permission, and regenerating without a change leaves the plan empty.

Undeclared use (T2, T2b). In Encore the edit gives the query component a reference of type `BucketRef[Uploader]` to the staging bucket and a write through it; T2b is the same edit. The model gains a write by the query component on the staging bucket, the export compiled from it gains `s3:PutObject`, and no warning is raised. In Nitric the model does not change in either scenario. In T2 the handle of the query component, created without `allow`, has no `file()` method, so the request fails with HTTP 500 and the first signal comes at run time. In T2b the handle is declared read-only, yet `nitric start` let the write through, and nothing reports that the model and the code disagree.

Leftovers (T4, T5). In T4 the declaration (Nitric) or the reference (Encore) of the archive bucket remains after the write call is removed, and the model and the export keep the write permission. The T4 plan equals the T1 plan. In T5 a topic with a subscriber and no publisher reaches the model and the export, and the plan creates the topic and its consumer. No stage log has a warning or error line that the T0 log lacks, and a search of the 56 log files of the run for *unused*, *dangling*, *orphan*, *no publisher* and *not referenced* finds nothing.

Rename (T6). Renaming the staging bucket re-points its uses to the new name in both tools. The Terraform address of the bucket changes, so the plan deletes the old bucket and creates a new one. The Nitric plan has 11 changes: the bucket, its random id and three policies are deleted and created again, and one parameter is updated. The plan of Encore's export has 6: the bucket is deleted and created again, and four IAM policies are updated. No log mentions the

rename. Nitric's bucket module has no setting that prevents deletion, and Encore documents none.

5 pyifc

5.1 Design

pyifc is a minimal annotation-based IfC tool for Python (Figure 2). The developer declares the resources and the capabilities with which each component uses them. An analyzer checks these declarations against the code and builds the model of the application, and a generator turns the model into Terraform for AWS. Access is both declared and derived from the code, and the two are compared. Permissions are generated only for capabilities the code uses. A lock file keeps the identity of each resource, so that a rename does not become a deletion.

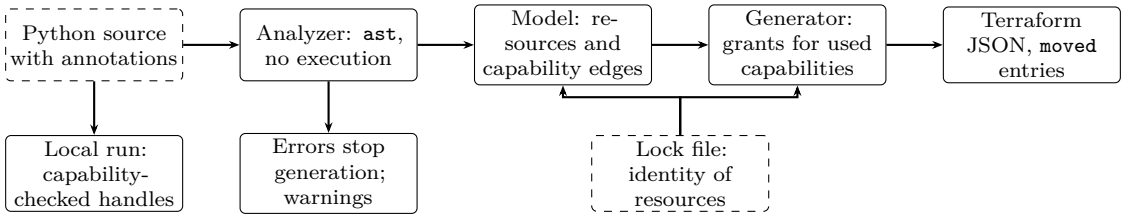


Figure 2: pyifc. Dashed boxes are files the developer keeps. The commands are `check`, `graph`, `generate` and `run`.

Annotations. Resources (`Bucket`, `KeyValueStore`, `Queue`, `Topic`, `Schedule`, `Secret`) are module-level objects with a literal id, and components are decorated functions: an HTTP handler, an event handler triggered by a topic or queue, or a scheduled job. A component states each use of a resource with a capability decorator: `read`, `write`, `list`, `delete`, `publish` or `subscribe`. The listing below shows the ingestion component of the sensor hub, abridged to one of its two topics.

```
staging = Bucket("sensorhub-staging")
speed   = Topic("sensorhub-speed")

@app.handler(method="POST", path="/readings", id="ingestion")
@app.write(staging)
@app.publish(speed)
def ingest_reading(body, content_type):
    ...
    staging.put(key, body)
    speed.publish(message)
```

Analyzer. The analyzer parses the source with `ast` and never runs it. It accepts a subset of Python: resources assigned once at module level and used by direct calls inside decorated top-level functions. It rejects aliases, `getattr`, resources passed to helpers, use outside a component and direct imports of `boto3`. Within the subset, it reports every direct call on a resource that no declared capability of the component covers. Each edge of the model records whether the capability was declared, used or both, or is implied by the runtime. Errors stop generation: a use without a declaration, a subscriber without a publisher, two ids that map to one name, a changed kind or an ambiguous rename. Warnings do not: a capability or a resource declared and never used, and a publisher without a subscriber.

Permissions. Every component has its own IAM role. The role grants the capabilities the analyzer sees the component use, the permission for its subscription trigger, the extra actions its

runtime adapters need (the *implied* edges) and a fixed logging statement. A capability that is declared and never used raises a warning and gets no permission, so the capabilities granted to a component are a subset of those it declares.

Identity and state. A lock file next to the application records, for each resource, its kind, its physical name and its use signature (the set of component and capability pairs that use it). The logical id is the Terraform address, and the physical name comes from the lock. When a resource disappears and another of the same kind appears with equal properties and use signature, pyifc infers a rename: it keeps the physical name, emits Terraform `moved` entries for the resource and its dependents, and warns. With several candidates, or a changed signature in the same edit, it refuses and lists the candidates, and the developer states the rename with `Bucket("new", was="old")`. Removing a stateful resource needs `--destroy <id>`.

Generation and local run. The generator emits Terraform JSON with one Lambda function per component and gives stateful resources `prevent_destroy`. The Lambda package is built deterministically and its hash is part of the export, so a change to the source shows in the plan as a function update. Per-resource overrides are limited to an allowlist (tags, versioning, queue timings) and cannot change permissions. `pyifc run` executes the application locally in one process. Each component receives handles that expose only the methods of its declared capabilities, and a call outside them raises `CapabilityError`.

5.2 Evaluation

I ran the eight scenarios on a Python port of the sensor hub for pyifc, with equivalent edits, the same pass rules and Terraform version, and a baseline applied to Floci in the same way. The edit stage is `pyifc check`. pyifc passes all eight scenarios (Table 4).

ID	Result	First signal	Terraform plan
T0	pass (reference)	None	No changes.
T1	pass (reference)	None	Creates the bucket; updates the ingestion policy and function.
T2, T2b	pass (signal)	Error `E_UNDECLARED_USE` at edit; generation refused	None, since generation stopped.
T3	pass (reference)	Warning `W_UNUSED_RESOURCE` at edit	Creates only the bucket.
T4	pass (signal)	Warning `W_UNUSED_CAPABILITY` at edit; no permission generated	Creates the bucket; updates one function; no policy change.
T5	pass (signal)	Error `E_TOPIC_MISSING_PUBLISHER` at edit; generation refused	None, since generation stopped.
T6	pass (signal)	Warning `W_RENAME_INFERRED` at generation	Updates three functions; the bucket is unchanged.

Table 4: pyifc: verdict, first signal and plan per scenario. *Reference*: passed because the model matches the reference. *Signal*: passed because pyifc raised a signal.

T0 and T1 raise no signal and give the same result as in Nitric and Encore. In the other six scenarios pyifc raises a signal at edit or, for T6, at generation, before any plan. The signals come from the comparison of declared and used capabilities (T2, T2b, T3, T4), the check that every subscribed topic has a publisher (T5) and the lock file (T6).

The errors in T2, T2b and T5 stop generation, so these scenarios have no plan. With the check disabled, `pyifc run` still raises `CapabilityError` on the write of T2b, which `nitric start` let through.

The warnings in T3, T4 and T6 let generation continue. In T3 the warning names the bucket that is declared and unused. In T4 the model keeps the declared write, as in the other tools, but

the generated policy leaves it out, so the T4 plan differs from the T1 plan. In T6 the warning suggests `was='sensorhub-staging'`. The bucket keeps its physical name and gets a `moved` entry, so the plan contains no destroy; the three function updates come from the changed source.

6 Discussion

On the sensor hub, both tools keep code and infrastructure aligned as long as the code and its declarations change together. The scenarios in which they part show that the outcome depends on where a tool keeps the information about access, and on whether anything compares that information with the code.

Encore keeps it in the code itself. A typed reference is both the way a service reaches a bucket and the statement of what the service may do with it, so a write through a reference always appears in the model, and Encore passes T2 and T2b. The same property lets access grow unnoticed. Adding a reference and a write is enough to give the query component write access to the staging bucket, and no step asks whether it should have it. Removing the call does not take the access away, because the reference stays (T4). Alignment with the code therefore does not give least privilege: in Encore, least privilege rests on a reviewer reading the reference types.

Nitric builds its model from declarations that sit next to the calls, and nothing compares the two. The run-time failure in T2 is a side effect of the Python SDK, where a handle without `allow` lacks the file methods. When the declaration exists but is too narrow (T2b), the local run lets the write through.

For the leftovers and the rename, neither tool looks at what the code no longer does. These changes become visible only in the plan, where they are easy to overlook: the plan of T4 equals that of T1, the plan of T5 creates the new topic like any other resource, and the deletion of the bucket in T6 is one of 11 changes in Nitric. A reader of the plan finds them only by knowing what to look for. In both exports the Terraform address of a resource comes from its name in the code, so a new name reads as a deletion followed by a creation.

pyifc combines declaration and derivation. Its declarations are a second description of access, the kind the introduction says can drift, but the analyzer compares them with the uses it derives from the code, so drift becomes an error or a warning instead of a silent difference. The declaration states the access the developer intends, and the derived uses decide what is granted, so neither can widen the permissions alone. The cost lies in writing and in expressiveness. The sensor hub port needs 15 decorator lines for its four components, and the analyzer accepts only a subset of Python. Deriving access from the code alone, as Encore does, needs no such lines but gives no point at which intended and actual access can differ.

Building pyifc also showed that not every mismatch calls for the same response. A use without a declaration would fail when the code runs, and a subscriber without a publisher would never run, so both stop generation. A leftover permission or a rename leaves a working application, so pyifc warns and continues, with the narrower policy or the kept name. Identity across a rename needs information the code does not hold, which pyifc keeps in the lock file.

7 Limitations

- *Application.* The comparison uses one small application with HTTP handlers, topic subscribers, buckets and topics, in one port per tool. Databases, queues, schedules and secrets are not exercised. Some results depend on the port: the run-time failure of Nitric in T2 comes from the handle of its Python SDK. Wing, Klotho and Shuttle were not run.
- *Scenarios.* The eight scenarios are single small edits, run once per tool by one evaluator. Direct cloud SDK calls, a declaration in an untaken branch and a shared helper package are not covered.

- *Deployment.* Nothing was deployed to AWS. The baseline was applied to Floci, and each change was only planned against it, without refreshing the state, so neither the enforcement of IAM policies nor the effect of a rename on a bucket that holds data was observed. Encore's export and plan come from my compiler; Encore Cloud's own provisioning was not run.
- *pyifc.* The analyzer accepts a subset of Python, and a service without a pyifc resource type cannot be used. The check that every use is declared assumes the program runs no code the analyzer cannot see, such as `exec` or an SDK loaded by name. A topic whose publisher lives outside the application is rejected. After a rename the physical name differs from the name in the code, and only the lock file links the two. An inferred rename is accepted with a warning, so deleting a bucket and adding a new one of the same kind and use would be read as a rename. pyifc targets AWS through Terraform only, and does not model regions, quotas, cold starts or cost.

8 Conclusion

Nitric and Encore carry a change from the code to the infrastructure when the code and its declarations change together. When the two diverge, neither tool reports it: a permission without a use, a subscriber without a publisher and a rename that deletes a bucket pass without a warning, and in Encore a new reference widens access silently. pyifc, a minimal annotation-based tool for Python, compares declared and derived access before it generates Terraform for AWS. In these scenarios it raised a signal before the plan wherever Nitric or Encore missed one, which shows that the gaps of both tools can be caught by checks run before generation.

Further work could deploy the generated infrastructure to AWS, to observe IAM enforcement and the effect of a rename on a bucket that holds data, and extend the scenarios to queues, databases and shared helper code. pyifc could accept helpers that receive a resource and gain adapters for other clouds and deployment targets. Language models could help where static analysis stops, by proposing declarations for code outside the accepted subset or by flagging access wider than the code needs, with the analyzer checking each proposal before generation. The same checks could also be tried on the models of Nitric and Encore, to see whether existing IfC tools can close these gaps without a new tool.

Acknowledgement

Generative AI (Claude Code, Anthropic) assisted in the spec-driven development [19] of the sensor hub and of pyifc, in porting the sensor hub to Nitric and Encore and fixing bugs in both ports, and in text editing and note-taking. I directed the work and reviewed its results.

References

[1] G. Simhandl and U. Zdun. "Cloud Programming Languages and Infrastructure from Code: An Empirical Study". In: *Proceedings of the 17th ACM SIGPLAN International Conference on Software Language Engineering (SLE '24)*. 2024, pp. 143–156. DOI: 10.1145/3687997.3695643. URL: https://doi.org/10.1145/3687997.3695643 (visited on 10/26/2026).

[2] *What is Terraform?* HashiCorp. URL: https://developer.hashicorp.com/terraform/intro (visited on 10/27/2026).

[3] *How Pulumi IaC works.* Pulumi Corporation. 2026. URL: https://www.pulumi.com/docs/iac/guides/basics/how-pulumi-works/ (visited on 10/27/2026).

[4] *AWS CloudFormation User Guide: What is CloudFormation, template format, template sections and resources syntax*. Amazon Web Services. 2026. URL: https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/template-formats.html (visited on 10/27/2026).

[5] I. Aviv, R. Gafni, S. Sherman, B. Aviv, A. Sterkin, and E. Bega. "Infrastructure From Code: The Next Generation of Cloud Lifecycle Automation". In: *IEEE Software* 40.1 (2023), pp. 42–49. DOI: 10.1109/MS.2022.3209958. URL: https://doi.org/10.1109/MS.2022.3209958 (visited on 10/26/2026).

[6] V. Tankov, D. Valchuk, Y. Golubev, and T. Bryksin. "Infrastructure in Code: Towards Developer-Friendly Cloud Applications". In: *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 2021, pp. 1166–1170. DOI: 10.1109/ASE51524.2021.9678943. arXiv: 2108.07842. URL: https://arxiv.org/abs/2108.07842 (visited on 10/26/2026).

[7] *Infrastructure from Code*. URL: https://www.infrastructurefromcode.com/ (visited on 09/26/2026).

[8] J. Jackson. *Infrastructure From Code: What Went Wrong*. The New Stack. June 2025. URL: https://thenewstack.io/infrastructure-from-code-what-went-wrong/ (visited on 10/26/2026).

[9] *Wing: Preflight and Inflight*. Wing. URL: https://www.winglang.io/docs/concepts/inflights (visited on 10/26/2026).

[10] *Project status*. Wing. URL: https://www.winglang.io/contributing/status (visited on 11/04/2026).

[11] *Klotho*. Klotho. URL: https://github.com/klothoplatform/klotho (visited on 10/26/2026).

[12] C. Hadjiaslanis. *DevLog[0]: Building a Serverless Platform for Rust in 4 Weeks*. Shuttle. Apr. 2022. URL: https://www.shuttle.dev/blog/2022/04/22/dev-log-0 (visited on 10/26/2026).

[13] *Shuttle shutdown*. Shuttle. URL: https://docs.shuttle.dev/docs/shuttle-shutdown (visited on 11/04/2026).

[14] *Why Nitric*. Nitric. 2026. URL: https://nitric.io/docs/get-started/foundations/why-nitric (visited on 10/26/2026).

[15] *Providers*. Nitric. URL: https://nitric.io/docs/providers (visited on 10/26/2026).

[16] *Understanding Encore*. Encore. URL: https://encore.dev/docs/understanding-encore (visited on 10/26/2026).

[17] *Self-hosting: build a Docker image*. Encore. URL: https://encore.dev/docs/self-host/build (visited on 10/26/2026).

[18] *Railigent X Train API: Overview*. Siemens. URL: https://developer.siemens.com/railigent-x/Train-API/index.html (visited on 11/05/2026).

[19] *GitHub Spec Kit: a specification-driven development toolkit*. GitHub. URL: https://github.github.com/spec-kit/ (visited on 11/05/2026).